

Requirements for Capability Broker

Adrian Olbricht ¹[0009-0006-5009-6591] , Jennifer Skopac ¹[0009-0007-5846-4640] ,
Jan Lundberg ^{1,2} [0009-0008-0485-6005] , Jelena Zdravkovic ¹[0000-0002-0870-0330]
and Janis Stirna ¹[0000-0002-3669-832X] ,

<u>Requirement category</u>	<u>Requirements</u>
1. Input text	Input can be in the form of text and can contain an organization description, processes, and prompts.
	The system shall be able to support the following types of input: - A textual prompt such as “Generate a capability map based on the description in this attached file.” - A follow-up textual prompt, such as "Can you integrate goals into the capability description?" - A textual description of an organizational function(s) or process, such as an overview of the process of target development within Joint Targeting (AJP 3.9). - Textual instructions on how to structure the diagram that is requested for the output, such as what the top-level capability should be. - Textual instructions on how detailed the diagram should be, such as the number of capability levels the diagram should specify. - PlantUML code for the agent to iterate, for example, in case of syntactical errors, missing elements and/or levels of the diagram.
2. File uploads	Input for the purpose of creating a knowledge repository that focuses on the LLMs' response.
2.1 Basic upload functions	The system should be able to identify user-uploaded files in chat as material to analyze and already-defined library files as the knowledge base it uses to analyze. The system should support formats such as PDF/JPEG/PNG/DOCX. The purpose is to analyze existing organizational information, models, and rich-text pictures. The system should enforce a check on the file type the user uploads, ensuring it is supported. The system should prevent uploading files that are too large. We want to ensure the user doesn't upload a file that is too “broad,” which could cause the agent to lose focus. The limit should be 10 MB.

	The system should enforce a limit on the number of library files the user can have in the knowledge base at any given time. The limit should be 5 files. The system should enforce a limit on the number of attached files per request. The limit should be 3 files. The system should enforce a check on the file type the user uploads, ensuring it is supported.
2.2 Creating a knowledge base and supporting RAG	The system should store domain-specific documents to specify context and use it as a knowledge base to support Retrieval Augmented Generation (RAG). For example, in the military domain, the base library should already include the NAFv4 specification, AJPs, or other relevant external knowledge. The system should support using meta-models as a framework for creating and representing the output diagram. For example, in the military domain, NAFv4 defines metamodels, definitions, and examples of the modelling language. This is then used by the agent as a rule set when creating the output diagram. The system should support the user in updating (such as adding or removing files) the knowledge base (library) with additional knowledge sources, such as AJP documents, to assist the agent in identifying, for example, important relationships between capabilities, processes, and activities. The system should allow the user to view the library files it currently uses as a knowledge base in a list format. The system should enforce a limit on the number of library files the user can have in the knowledge base at any given time. The limit should be 5 files.
3. Capability map creation	These requirements focus on the functionality for creation of capability map.
3.1 Start a new/restart conversation	The system should support the user to restart the chat with the agent, erasing the previous “context” of the conversation. The system needs to inform the user of the consequences of restarting the conversation, such as the deletion of all diagrams in the current chat.

3.2. Identify capabilities and activities.	The system should identify fragments of the input text as activities and capabilities based on predefined definitions of the concepts, contained in the agent's instructions. The system should be able to map sub-capabilities and sub-activities across different levels and accurately represent their relationships.
3.3 Instructions on how to use best practices.	The system should have a textual instruction available that describes how to best prompt the agent and how to use the application generally.
3.4. Have preset questions/prompts.	The system should provide users with preset prompts known to yield good results. The system should provide the prompts in a list format displayed next to the chat window for easy access. The system should provide the user with the possibility to save prompts as best practice.

Requirements for Capability Map display

4. Capability Map Display	
4.1 Save the diagram image	The system should produce a file containing the diagram image, which the user can save for viewing. The system should provide the diagram as a PDF file.
4.2 Export PlantUML code	The system should generate syntactically and semantically correct PlantUML code in response to the user's request. The system should present a copyable code extract of the PlantUML that generated the diagram. The system should distinguish between classes that represent capabilities and those that represent activities. For example, by labelling them in different colours. The system should be able to transmit user input to the LeChat API for PlantUML generation.
4.3 Display the capability map	The system should be able to display the capability map to the user.

4.4 Be able to send PlantUML code and receive a file via API.	The PlantUML code returned by the LeChat API should be sent to an external diagram generation tool before being presented to the user. The diagram created by the external tool should be received as a PDF
4.5 Save the history of the session PlantUML code	The system should save the history of the current conversations' PlantUML code, so it can be appended to the request if the user wants to make changes to a specific iteration of a diagram.
4.6 Save chat history	The system should save the current conversation history of prompts and responses. Outputs should be saved as context for the agent. The system should display the conversation history to the user in chronological order, with the oldest messages at the top.
4.7 Handle faulty code produced by the agent.	The system needs to handle situations where the agent responds with non-compiling PlantUML code. This can be communicated to the user, prompting them to try again. Alternatively, retry behind the scenes until it works.

Requirements for capability expansion

5. Capability expansion	
5.1 Match capabilities to other existing capabilities.	The system should be able to match newly identified capabilities to already-defined capabilities based on user input, identifying which ones support each other.
5.2 Match capabilities to other existing processes.	The system should be able to match newly identified capabilities to an already-defined process based on user input, identifying which ones support each other.
5.3 Iterate the process through a chat function.	The system should be able to re-generate the output diagram, taking into account the specified feedback from the user in a back-and-forth process. For example, if the user wants certain classes to be renamed in the diagram. The system should take previous messages into account as context when generating the response.